

AI GENERALIST ACCELERATOR · 14-DAY AI SPRINT

Day Eight

Front Ends for Your Workflows: Webhooks, Lovable, and GitHub

A cleaned, detailed record of the live session and breakout briefing

Topics covered: APIs and Webhooks · GET Versus POST · Front End, Back End, and the Bridge · The Support Form in Lovable · Secrets, Test and Production URLs · Respond to Webhook · GitHub as Google Drive for Code · Branches, Pull Requests, and Merging · The Workbook

About This Session

Week two opened with the answer to the question the cohort had asked all through week one: the workflows built on days three, four, and seven all run only inside n8n, triggered by their builder, and nobody is going to give customers a login to their back end, so how do you put one on a website and share it? Shasti framed the evening around exactly that, a front end for any workflow so its link can be handed to anyone, plus a first introduction to GitHub, flagged as essential because the hackathon opens on day eleven and teams will need to share and collaborate on a codebase. The mentor was Robert Breen, an n8n ambassador and former Tableau ambassador with fifteen years in data science, who runs Interactive, an AI-automation training and data-science company for small and medium businesses, and who most of the room had already met in a week-one breakout. His promise was an end-to-end product in two hours, the same one everybody would rebuild in the breakout, so the teaching was for listening only, and his method was Socratic throughout, every concept drawn on the board and voted on in the chat before it was named.

HOW TO USE THESE NOTES

The workbook rebuilds the day-four customer-support triage bot in n8n and then adds the Lovable front end exactly as demonstrated, step by step, with co-mentors in the rooms; it is designed to complete on Lovable's free tier if you stick to the given prompts and models, and the OpenAI API key in the workbook stays active until 6 pm IST the next day. If your free Lovable credits are spent, sign in with an alternate email address, since credits are not something Outskill can solve.

Content and workbook questions go to the Ask Doubt channel on the platform, where a dedicated mentor replies within 24 hours; the support email is only for logistics, bonus tools, and schedule. The day-eight skill graph recheck is live, using your registered email to find your personal link.

APIs and Webhooks: The Pizza Restaurant

Robert began with the word most of the room feared and stripped it down. An API, application programming interface, is simply how applications talk to each other, a bridge, as someone in the chat put it, and you are doing programming now because you use AI, so the term should intimidate nobody. Google is the everyday example: search for a plumber near you and Google uses an API to fetch that information and return it, as does virtually every app on your phone. Then the real-world problem, told as a story from before food-delivery apps existed. You are on the couch wanting fifteen pizzas for the team, so you call the restaurant and order; an hour later nothing has arrived, so you call again, and again, and again until the pizza comes or you give up, which is scenario one, calling the restaurant, an active way of getting information. The better way, which the chat found immediately, is scenario two: the restaurant calls you when the food is ready, a passive way of getting information. The names followed the votes. The active way, Google going to fetch what you asked for with nobody calling Google, is an API; the passive way, sitting and waiting to be told, is a webhook, both doing the same job in opposite directions. For the restaurant to call you it needs three things, one of them optional: your phone number, which in programming is a URL; authorisation, optional, the equivalent of telling the restaurant it is allowed to call you rather than being cold-called at midnight; and a method, either POST or GET, defined a minute later. GET gets information, Google searching or an app reading your email; POST, which he wished had simply been called send, sends information, an app sending an email. And the day's form, name, email, feedback, and a submit button, is a POST, because the form fetches nothing, it sends what was typed to the back end.

Front End, Back End, and the Bridge Between Them

Every application has two parts, a front end that the consumer sees and a back end that does the work, with a database part of the back end. The room already knew tools for both, n8n and SQL for back ends, Lovable and Bolt for front ends, and Robert's rule was not to worry about the specific tool, since the concepts never change. The example was teed up as a support form: name, email, feedback, submit, and when submit is clicked the data must move to the back end and something must happen there, possibly several things, a row in Google Sheets, an email, an entry in another tool. Front ends are pretty, Lovable makes very pretty websites, but they do little; the back end does the work. Between them sits the webhook, and the trivia question of what the webhook needs to send the form's data returned the three answers already learned, a URL as the phone number, a method, which is POST because the front end is sending, and optional authorisation. The two existing workflows from days three and four were pinned to the board, the spam-checking form submission and the chat-triggered support agent that writes to Google Sheets and sends emails, both back ends, and the connection was drawn in: submit goes to a webhook, and the webhook goes directly to the AI agent, bypassing the chat trigger entirely, because a webhook is, in his phrase, a backwards API.

The Build: Webhook Trigger, Lovable Form, and Secrets

In n8n he first copied the entire day-four workflow to keep the original safe, noting that a copied workflow never carries its chat trigger since only one can exist, and that memory is optional here because memory is not intelligence, only recall of the last thing said. The trigger became a webhook node, which arrives with two phone numbers, a test URL and a production URL, a path he renamed to feedback, authentication left off as optional, and the method switched from GET to POST, a step he confessed to missing himself in recent months and losing an hour to, because a webhook receiving a form is sending information onward. That was the back end done. For the front end the chat proposed Lovable, Bolt, Claude Code, plain HTML, and Cursor, all correct, and Lovable was chosen. The prompt asked for a clean, modern customer-support feedback form as a single landing page with name, email, feedback, and a submit button that sends the data via POST to the webhook URL, with the sentence he asked everyone to hear: the webhook URL will be added under a secret config. The reasoning was the security lesson of the night, would you type your password into Lovable, would you print your phone number on a billboard, so the webhook URL, which is both, never goes to the AI; it goes to the application through a secret config, also called an ENV file, greyed out and never visible to Lovable's model, exactly where API keys and passwords belong too. When Lovable asked for the feedback URL, the test URL went in. The first submission failed with a 404, and the room diagnosed it: a test URL only works while the workflow is listening, so Listen for Test Event turned it on and the form returned a thank-you while n8n showed the name, email, feedback, and timestamp arriving. One edit made the agent work: its user message still referenced a chat input that no longer existed, shown in red as n8n marks any missing field, so the three webhook fields, name, email, and feedback, were dragged in as expressions, in any order, since agents do not care about formatting, and a second submission asking for dark mode ran the whole day-four pipeline, OpenAI, Google Sheets, Gmail, and delivered the automated acknowledgement to his inbox.

Going Live: Production URLs, Executions, and Respond to Webhook

Nobody wants to click Execute Workflow every time a form arrives at midnight, which is what the production URL is for, always on and always waiting, whereas the test URL must be turned on for each test. Publish in n8n's top corner, not to be confused with save, exposes the webhook to the live internet, on free plans too, and the next submission failed anyway, because Lovable still held the test URL, the kind of mistake that genuinely happens in the real world; the only difference between the two addresses is that the test one says webhook-test where production says webhook. Changing it meant another refusal to hand a production URL to an AI agent, and instead a written procedure in Lovable, More, then Cloud, then Secrets, where the secret Lovable had named Feedback Webhook URL cannot be edited, so it is deleted and recreated under exactly the same name, because Lovable uses that name as a variable, with the production URL as its value. A no-dark-mode submission then landed in the sheet and sent its email without any button being pressed, and the Executions tab showed how to verify it, test runs marked with a symbol, production runs without. We are live, he said, it is deployed, that is what deploying means. One improvement remained: a form that answers thank you, your feedback has been submitted, wastes the generative AI behind it. The webhook node's response setting was changed from respond immediately to using a Respond to Webhook node, added at the very end of the workflow, the easiest node in the world, so that n8n answers the phone with a got-it, does its job, and only then delivers its output, and Lovable was told in plain language to show the custom response after submission. A can't-log-in complaint came back with the customer's name, email, a feedback summary, and the actions taken, logged and acknowledged by email, all in the execution log at 11.35. An application that would once have taken a developer a day was built on a call in ninety minutes, published for the room to try to break during the break, and unpublished afterwards because n8n plans carry monthly run limits.

GitHub: Google Drive for Code

The application just built is code, thousands of lines Lovable wrote behind the scenes, and code needs a place to be stored securely, revised, and worked on together, which is GitHub, used by virtually every large company since around 2008. The reasoning went through Salesforce and Google: hundreds or thousands of engineers working on the same code at the same time across countries, time zones, and home offices need a shared store, a way to collaborate on identical code simultaneously, and versioning, so that when version two breaks, and people and AI both make mistakes, version one can be restored, with backup as the bonus. The comparison that defused it was Google Drive: Drive stores files and folders and even keeps a rudimentary version history; GitHub does the same for code with different words, where a folder is called a repository and every change is kept as history. n8n itself lives there as open source, its entire product public, so he opened the repository and read the day's commits, a named employee's change from twenty-six minutes earlier visible line by line, unchangeable by outsiders but free to read and reuse, beside his own private client repositories. You do not need a software background for any of it; think of it as Google Drive for the code your vibe-coding tools produce. Connecting was three clicks in Lovable, More, Settings, Git, with a one-time GitHub sign-in, after which Lovable created a repository with a cheerful auto-generated name and the code appeared on GitHub, its Commits view revealing what nobody notices while waiting for Lovable, the small additions and removals it made for each prompt, the feedback form, the webhook. GitLab and Bitbucket do the same job; only the front-end code is stored here.

Branches, Pull Requests, and How Teams Actually Ship

Publishing to GitHub is not the end of the process. Every application runs on a main or master branch, and nobody at Google opens the live product and edits it on a whim; instead a developer takes a copy of the code, a branch, makes changes there while colleagues branch and change in parallel, and when finished submits a pull request, which means simply I am done, after which the changes are merged back into main, illustrated on the board and then with a small Claude-built animation. The nine steps for the hackathon followed: the team lead creates the Lovable app and gives the team access; each member branches, makes changes, tests them, and submits a pull request, a PR, only once the change works; the team lead tests the request and merges. The live demonstration made it concrete. His team lead wants dark mode, and editing the running site directly is out, so under More and Git, Create Branch made a dark-mode branch, with the collaboration rule stated: never work on main. Add a dark mode ran on that branch, light and dark toggled in the top corner, and, as team member, he opened Compare and Pull Request on GitHub, described the change as done and tested, and created the request, which found no conflicts. Switching hats to team lead, he tested the change, found it exactly as described, and clicked Merge Pull Request, and the whole cycle, member branches, member tests, member requests, lead tests, lead merges, was complete. This, he closed, is how software is built at virtually every company.

THE MENTAL MODEL

An API fetches, a webhook waits; a webhook needs a URL, a method, and optional authorisation, and a form that sends is a POST. Front ends are pretty and back ends do the work, joined by a webhook that replaces the chat trigger. Keep URLs and keys in secrets, never in prompts; test, publish, switch to production, read the Executions tab. Code lives in GitHub like files in Drive; never work on main, branch, test, pull-request, and let the lead merge.

The Skill Graph, the Workbook, and What Comes Next

Before the rooms, Shasti brought back the day-one skill graph, enter your registered email and answer the same questions to see how far seven days have moved the score. The workbook rebuilds the day-four triage bot and adds the Lovable interface exactly as Robert had, and the rooms ran over two hours until most of the chat reported a first Lovable interface connected to n8n. Tomorrow's schedule was set: office hours at 6 pm IST, then the day-nine session at 7.30 with Dileep back for open-source and local models plus a Codex setup.

Key Takeaways

Stripped to its essentials, the session made the following points.

- **An API fetches; a webhook waits.** Calling the restaurant over and over is an API; the restaurant calling you when the pizza is ready is a webhook, moving the same information in opposite directions.
- **A webhook needs three things.** A URL as its phone number, a method, and optional authorisation; GET gets information and POST sends it, so a submit button is always a POST.
- **Front ends are pretty, back ends do the work.** Any tool serves either side, and connecting them means swapping the chat trigger for a webhook that feeds the agent the form's fields as expressions.
- **Never give URLs or keys to the AI.** Webhook URLs, API keys, and passwords go into the application through a secret config, invisible to Lovable's model, exactly like an ENV file.
- **Test, publish, switch, verify.** The test URL works only while listening; publish, replace the secret with the production URL under the same name, and read the Executions tab.
- **Respond to Webhook makes the form intelligent.** Respond through the node at the end instead of immediately, and the form returns the agent's summary and actions instead of a canned thank-you.
- **GitHub is Google Drive for code.** Folders are repositories, history is versions, n8n's own code is public there, and Lovable connects in three clicks.
- **Branch, test, pull-request, merge.** Never work on main; each teammate branches for every change and requests a merge only after testing, and the team lead tests and merges.

End of Day Eight notes. Next: open-source models and Codex — Day Nine.